

1 MELHORIAS DE ENGENHARIA DE SOFTWARE

A versão final apresentou:

- arquitetura modular;
- separação por classes;
- dataclasses;
- validação automática;
- tratamento de exceções;
- código reutilizável;
- melhor organização estrutural.

2 COMPARAÇÃO ENTRE AS VERSÕES

Critério	Primeira Versão	Versão Final
Física orbital	Simplificada	Avançada
Método numérico	Euler	RK4
IA	Linear simples	Ridge + Polinomial
Atmosfera	Exponencial simples	Multicamada
Monte Carlo	Não	Sim
Visualização 3D	Básica	Avançada
Dashboard	Simple	Científico
Heatmap	Não	Sim
Gauge	Não	Sim
Exportação CSV	Parcial	Completa
Robustez	Média	Alta
Organização	Intermediária	Profissional

3 CÓDIGO FINAL

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
```

```
"""
```

```
SISTEMA DE ANÁLISE DE DECAIMENTO ORBITAL
=====
```

```
Simulação física de satélites em órbita baixa (LEO).
```

Modelos implementados:

- Atmosfera NRLMSISE-00 simplificada (escala de altura)
- Perturbação J2 (achatamento terrestre)
- Integração RK4 com passo adaptativo (Richardson extrapolation)
- Análise de incerteza Monte Carlo
- Predição por regressão polinomial

Uso:

```
python decaimento_orbital.py          # interativo
python decaimento_orbital.py --preset LEO_500 # preset rápido
python decaimento_orbital.py --help
```

```
"""
```

```
import argparse
import sys
from copy import deepcopy
from dataclasses import dataclass, field
from datetime import datetime
from pathlib import Path
```

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import Ridge
from sklearn.pipeline import make_pipeline
from sklearn.metrics import r2_score
```

```
plt.style.use("ggplot")
```

```
#
```

```
# CONSTANTES FÍSICAS
```

#

```
@dataclass(frozen=True)
class Constantes:
    G: float = 6.67430e-11 # m³ kg⁻¹ s⁻²
    M_terra: float = 5.972e24 # kg
    R_terra: float = 6_371_000.0 # m
    J2: float = 1.08263e-3 # coef. achatamento
    g0: float = 9.80665 # m s⁻² (padrão ISA)
```

```
@property
def GM(self) -> float:
    return self.G * self.M_terra
```

CONST = Constantes()

#

PARÂMETROS DO SATÉLITE

#

```
@dataclass
class Satellite:
    """
    Parâmetros físicos do satélite.

    Atributos
    -----
    massa : kg
    area : m² (área de referência para arrasto)
    cd : adimensional (coeficiente de arrasto, tipicamente 2.0–2.5 em LEO)
    altitude: m (altitude inicial acima do nível do mar)
    """
    massa: float = 800.0
    area: float = 2.0
    cd: float = 2.2
    altitude: float = 500_000.0 # 500 km

    @property
    def bc(self) -> float:
        """Coeficiente balístico  $B^* = m / (C_d \cdot A)$  [kg m⁻²]"""
```

```

        return self.massa / (self.cd * self.area)

def validar(self):
    assert self.massa > 0, "Massa deve ser positiva"
    assert self.area > 0, "Área deve ser positiva"
    assert self.cd > 0, "Cd deve ser positivo"
    assert self.altitude > 0, "Altitude inicial deve ser positiva"

#


---




---


# MODELO ATMOSFÉRICO (NRLMSISE-00 simplificado)
#


---




---



# Tabela: altitude base (m), densidade base (kg m-3), escala de altura (m)
# Fonte: US Standard Atmosphere 1976 / NRLMSISE-00
_CAMADAS = np.array([
    [ 80_000, 5.99e-5, 6_369],
    [ 100_000, 5.60e-7, 7_657],
    [ 120_000, 2.08e-8, 9_473],
    [ 150_000, 2.07e-9, 12_762],
    [ 200_000, 2.54e-10, 18_974],
    [ 300_000, 1.92e-11, 26_789],
    [ 400_000, 3.97e-12, 31_830],
    [ 500_000, 1.06e-12, 37_105],
    [ 600_000, 3.21e-13, 45_854],
    [ 700_000, 8.77e-14, 53_632],
    [ 800_000, 1.48e-14, 68_646],
    [1_000_000, 5.15e-15, 88_667],
], dtype=float)

def densidade_atmosferica(h: float, F107: float = 150.0) -> float:
    """
    Densidade atmosférica pelo modelo de escala de altura.

    Parâmetros
    -----
    h : altitude (m)
    F107 : índice de atividade solar (unidades de fluxo, tipicamente 70–300)

    Retorna
    -----
    rho : densidade [kg m-3]
    """

```

```

# Guarda: altitude fora do domínio físico
h = float(h)
if not np.isfinite(h):
    return 0.0
h = np.clip(h, _CAMADAS[0, 0], _CAMADAS[-1, 0])

# Fator de correção solar (King-Hele, 1987)
F107 = np.clip(float(F107), 70.0, 400.0)
k_solar = 1.0 + 0.012 * (F107 - 150.0) / 50.0

# Encontra a camada correta
idx = int(np.searchsorted(_CAMADAS[:, 0], h, side='right')) - 1
idx = np.clip(idx, 0, len(_CAMADAS) - 2)

h0, rho0, H = _CAMADAS[idx]

# Clamp do expoente para evitar overflow (~±700 é o limite do float64)
exp_arg = np.clip(-(h - h0) / H, -700.0, 700.0)
rho = rho0 * np.exp(exp_arg)

return float(rho * k_solar)

```

#

DINÂMICA ORBITAL

#

class DinamicaOrbital:

"""Equações de movimento para órbita circular com arrasto e J2."""

def __init__(self, sat: Satellite, F107: float = 150.0):

self.sat = sat

self.F107 = F107

def velocidade_circular(self, h: float) -> float:

""" $v_c = \sqrt{GM / r}$ [m s⁻¹】"""

r = CONST.R_terra + max(h, 0.0) # r nunca negativo

if r <= 0:

return 0.0

return np.sqrt(CONST.GM / r)

def aceleracao_arrasto(self, h: float) -> float:

"""

$a_d = (1/2) * \rho * v^2 * C_d * A / m$ [m s⁻²]

Para órbita circular, $v = v_c(h)$.
Sinal negativo → desaceleração.

"""

```
rho = densidade_atmosferica(h, self.F107)
v = self.velocidade_circular(h)
if not (np.isfinite(rho) and np.isfinite(v) and v > 0):
    return 0.0
F = 0.5 * rho * v**2 * self.sat.cd * self.sat.area
if not np.isfinite(F):
    return 0.0
return -F / self.sat.massa
```

def aceleracao_j2(self, h: float) -> float:

"""

Perturbação radial J2 (componente secular na altitude).
Efeito principal: variação na altitude média ao longo da órbita.

"""

```
r = CONST.R_terra + max(h, 0.0)
val = -1.5 * CONST.J2 * CONST.GM * CONST.R_terra**2 / r**4
return float(val) if np.isfinite(val) else 0.0
```

def dh_dt(self, h: float) -> float:

"""

Taxa de variação da altitude [m s^{-1}].

Para órbita circular perturbada por arrasto:

$$dh/dt = (2r/v) * a_d + \text{correção_J2}$$

Ref: King-Hele (1987), "Satellite Orbits in an Atmosphere"

"""

```
r = CONST.R_terra + h
v = self.velocidade_circular(h)
ad = self.aceleracao_arrasto(h)
```

Variação de altitude por arrasto (derivação via energia orbital)

$$dh_arrasto = (2.0 * r / v) * ad$$

Contribuição J2 (pequena comparada ao arrasto em LEO)

$$dh_j2 = (2.0 * r / v) * self.aceleracao_j2(h) * 0.01$$

```
return dh_arrasto + dh_j2
```

#

INTEGRADOR RK4 + PASSO ADAPTATIVO (Richardson)

#

```
def _rk4_passo(f, y: float, dt: float) -> float:
```

```
    k1 = f(y)
```

```
    k2 = f(y + 0.5 * dt * k1)
```

```
    k3 = f(y + 0.5 * dt * k2)
```

```
    k4 = f(y + dt * k3)
```

```
    return y + (dt / 6.0) * (k1 + 2*k2 + 2*k3 + k4)
```

```
def rk4_adaptativo(
```

```
    f,
```

```
    y: float,
```

```
    dt: float,
```

```
    tol: float = 1.0,      # tolerância em metros
```

```
    dt_min: float = 600.0, # 10 min
```

```
    dt_max: float = 86_400.0, # 1 dia
```

```
) -> tuple[float, float]:
```

```
    """
```

```
    Passo RK4 com controle de erro por extrapolação de Richardson.
```

```
    Retorna (y_novo, dt_sugerido).
```

```
    """
```

```
    # Limite de iterações para evitar recursão infinita
```

```
    for _ in range(40):
```

```
        y1 = _rk4_passo(f, y, dt)
```

```
        y2 = _rk4_passo(f, _rk4_passo(f, y, dt / 2), dt / 2)
```

```
        erro = abs(y2 - y1)
```

```
        if erro < 1e-12: # praticamente zero — aumenta passo
```

```
            dt_novo = min(dt * 2.0, dt_max)
```

```
            return y2, dt_novo
```

```
        fator = (tol / erro) ** 0.2
```

```
        if erro <= tol:
```

```
            dt_novo = min(dt * 0.9 * fator, dt_max)
```

```
            return y2, dt_novo
```

```
        dt = max(dt * 0.9 * fator, dt_min)
```

```
    # Fallback: retorna com passo mínimo
```

```
    return _rk4_passo(f, y, dt_min), dt_min
```

```
#
```

```
# SIMULADOR PRINCIPAL
```

```
#
```

```
class Simulador:
```

```
    H_REENTRADA = 80_000.0 # altitude de reentrada [m]
```

```
    def __init__(self, sat: Satellite, F107: float = 150.0):
```

```
        sat.validar()
```

```
        self.sat = sat
```

```
        self.F107 = F107
```

```
        self.dinamica = DinamicaOrbital(sat, F107)
```

```
    def simular(
```

```
        self,
```

```
        anos_max: float = 30.0,
```

```
        dt_inicial: float = 86_400.0,
```

```
    ) -> pd.DataFrame:
```

```
        """
```

```
        Simula o decaimento orbital.
```

```
        Retorna DataFrame com colunas:
```

```
            tempo_s, tempo_anos, altitude_m, altitude_km
```

```
        """
```

```
        t = 0.0
```

```
        h = self.sat.altitude
```

```
        dt = dt_inicial
```

```
        t_max = anos_max * 365.25 * 86_400.0
```

```
        tempos = []
```

```
        altitudes = []
```

```
        f = self.dinamica.dh_dt
```

```
        while h > self.H_REENTRADA and t < t_max:
```

```
            tempos.append(t)
```

```
            altitudes.append(h)
```

```
            h, dt = rk4_adaptativo(f, h, dt)
```

```
            t += dt
```

```
        # Adiciona ponto final
```

```
        tempos.append(t)
```



```
t_arr = np.array(tempos)
h_arr = np.array(altitudes)
```

```
return pd.DataFrame({
    "tempo_s": t_arr,
    "tempo_anos": t_arr / (365.25 * 86_400.0),
    "altitude_m": h_arr,
    "altitude_km": h_arr / 1000.0,
})
```

#

ANÁLISE MONTE CARLO

#

@dataclass

```
class ConfigMC:
```

```
n_amostras:  int  = 200
sigma_massa:  float = 0.05 # desvio relativo (5 %)
sigma_area:   float = 0.10 # desvio relativo (10 %)
sigma_cd:     float = 0.08
sigma_F107:   float = 30.0 # unidades absolutas
anos_max:     float = 30.0
```

```
def monte_carlo(sat: Satellite, F107: float, cfg: ConfigMC) -> pd.DataFrame:
```

■■■■■

Amostragem Monte Carlo com perturbações nos parâmetros físicos.

Retorna DataFrame com estatísticas de vida orbital.

■■■■■

```
resultados = []
```

```
rng = np.random.default_rng(seed=42)
```

```
for i in range(cfg.n_amostras):
```

[illegible]

```

sat_i.cd      = max(0.1, rng.normal(sat_base.cd,
                                   cfg.sigma_cd * sat_base.cd))
F107_i = max(70.0, rng.normal(F107_base, cfg.sigma_F107))

```

```

try:
    sim = Simulador(sat_i, F107_i)
    df = sim.simular(anos_max=cfg.anos_max)
    vida = df["tempo_anos"].iloc[-1]
except Exception:
    continue

```

```

resultados.append({
    "amostra": i,
    "massa":   sat_i.massa,
    "area":    sat_i.area,
    "cd":      sat_i.cd,
    "F107":    F107_i,
    "vida_anos": vida,
})

```

```

return pd.DataFrame(resultados)

```

```

#

```

```

# PREDIÇÃO POR REGRESSÃO POLINOMIAL
#

```

```

class PreditorDecaimento:

```

```

    """

```

```

    Regressão polinomial Ridge para estimar tempo até reentrada
    a partir da altitude.

```

```

    """

```

```

    def __init__(self, grau: int = 4):
        self.modelo = make_pipeline(
            PolynomialFeatures(grau, include_bias=False),
            Ridge(alpha=1e-3),
        )
        self._treinado = False

```

```

    def treinar(self, df: pd.DataFrame) -> float:
        """Treina e retorna R²."""
        X = df["altitude_km"].values.reshape(-1, 1)
        y = df["tempo_anos"].values

```

```
self.modelo.fit(X, y)
self._treinado = True
y_pred = self.modelo.predict(X)
return r2_score(y, y_pred)
```

```
def prever(self, altitude_km: float) -> float:
    if not self._treinado:
        raise RuntimeError("Modelo não treinado. Chame .treinar() primeiro.")
    return float(self.modelo.predict([[altitude_km]])[0])
```

```
#
```

```
# VISUALIZAÇÃO
```

```
#
```

```
class Visualizador:
```

```
    @staticmethod
```

```
    def decaimento(df: pd.DataFrame, titulo: str = "Decaimento Orbital"):
        fig, ax = plt.subplots(figsize=(12, 5))
        ax.plot(df["tempo_anos"], df["altitude_km"], lw=2, color="steelblue")
        ax.axhline(80, color="red", ls="--", lw=1, label="Reentrada (80 km)")
        ax.set_title(titulo, fontsize=14)
        ax.set_xlabel("Tempo (anos)")
        ax.set_ylabel("Altitude (km)")
        ax.legend()
        ax.grid(True, alpha=0.4)
        plt.tight_layout()
        plt.show()
```

```
    @staticmethod
```

```
    def monte_carlo(df_mc: pd.DataFrame):
        fig, axes = plt.subplots(1, 2, figsize=(14, 5))

        # Histograma de vida
        axes[0].hist(df_mc["vida_anos"], bins=25, color="steelblue", edgecolor="white")
        mu = df_mc["vida_anos"].mean()
        axes[0].axvline(mu, color="red", ls="--", label=f"Média: {mu:.2f} a")
        axes[0].set_title("Monte Carlo — Vida Orbital")
        axes[0].set_xlabel("Anos")
        axes[0].set_ylabel("Frequência")
        axes[0].legend()
```

```
    # Sensibilidade: vida vs massa
```

```

sc = axes[1].scatter(
    df_mc["massa"], df_mc["vida_anos"],
    c=df_mc["F107"], cmap="plasma", alpha=0.6, s=20
)
plt.colorbar(sc, ax=axes[1], label="F10.7")
axes[1].set_title("Sensibilidade: Massa vs Vida")
axes[1].set_xlabel("Massa (kg)")
axes[1].set_ylabel("Vida (anos)")

plt.tight_layout()
plt.show()

```

```

@staticmethod
def densidade_atmosferica():
    h_vals = np.linspace(80_000, 800_000, 500)
    rho_150 = [densidade_atmosferica(h, 150) for h in h_vals]
    rho_250 = [densidade_atmosferica(h, 250) for h in h_vals]
    rho_70 = [densidade_atmosferica(h, 70) for h in h_vals]

    fig, ax = plt.subplots(figsize=(8, 6))
    ax.semilogy(h_vals/1000, rho_150, label="F10.7 = 150 (médio)")
    ax.semilogy(h_vals/1000, rho_250, label="F10.7 = 250 (alto)", ls="--")
    ax.semilogy(h_vals/1000, rho_70, label="F10.7 = 70 (baixo)", ls=":")
    ax.set_title("Modelo Atmosférico")
    ax.set_xlabel("Altitude (km)")
    ax.set_ylabel("Densidade (kg m-3)")
    ax.legend()
    ax.grid(True, which="both", alpha=0.3)
    plt.tight_layout()
    plt.show()

```

#

RELATÓRIO

#

```

def gerar_relatorio(
    df_sim: pd.DataFrame,
    df_mc: pd.DataFrame,
    sat: Satellite,
    F107: float,
    saida: Path = Path("relatorio_orbital.csv"),
):
    df_sim.to_csv(saida, index=False)

```

```

print("\n" + "=" * 60)
print(" RELATÓRIO — DECAIMENTO ORBITAL")
print("=" * 60)
print(f" Data/Hora      : {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")
print(f" Massa         : {sat.massa:.1f} kg")
print(f" Área          : {sat.area:.2f} m²")
print(f" Cd            : {sat.cd:.2f}")
print(f" Coef. Balístico : {sat.bc:.2f} kg m⁻²")
print(f" Altitude inicial: {sat.altitude/1000:.1f} km")
print(f" F10.7          : {F107:.0f}")
print("-" * 60)
print(f" Reentrada (det.): {df_sim['tempo_anos'].iloc[-1]:.3f} anos")
print(f" Vida MC (média) : {df_mc['vida_anos'].mean():.3f} anos")
print(f" Vida MC (σ)     : {df_mc['vida_anos'].std():.3f} anos")
print(f" Vida MC (P5–P95): "
      f"{df_mc['vida_anos'].quantile(0.05):.2f} – "
      f"{df_mc['vida_anos'].quantile(0.95):.2f} anos")
print("-" * 60)
print(f" Arquivo CSV     : {saida.resolve()}")
print("=" * 60)

```

#

ENTRADA DE DADOS (INTERATIVO / ARGUMENTOS)

#

```

PRESETS = {
    "LEO_300": Satellite(massa=500, area=1.5, cd=2.2, altitude=300_000),
    "LEO_500": Satellite(massa=800, area=2.0, cd=2.2, altitude=500_000),
    "LEO_700": Satellite(massa=1200, area=3.0, cd=2.2, altitude=700_000),
    "CubeSat": Satellite(massa=4, area=0.03, cd=2.5, altitude=400_000),
}

```

```

def entrada_interativa() -> tuple[Satellite, float, int, float]:
    print("\n" + "=" * 60)
    print(" SISTEMA DE DECAIMENTO ORBITAL — Configuração")
    print("=" * 60)

    print("\n Presets disponíveis:")
    for k, s in PRESETS.items():
        print(f" {k:10s} {s.altitude/1000:.0f} km, {s.massa:.0f} kg, "
              f"fBc={s.bc:.1f} kg/m²")

```

```

print(" (enter para digitar manualmente)")

preset = input("\n Escolha preset ou enter: ").strip().upper()
if preset in PRESETS:
    sat = deepcopy(PRESETS[preset])
    print(f" ✅ Usando preset {preset}")
else:
    print("\n --- Parâmetros do satélite ---")
    sat = Satellite(
        massa = float(input(" Massa (kg) [800]: ") or 800),
        area = float(input(" Área de referência (m²) [2.0]: ") or 2.0),
        cd = float(input(" Coeficiente de arrasto Cd [2.2]: ") or 2.2),
        altitude = float(input(" Altitude inicial (km) [500]: ") or 500) * 1000,
    )

F107 = float(input("\n Atividade solar F10.7 [150]: ") or 150)
n_mc = int(input(" Amostras Monte Carlo [200]: ") or 200)
anos = float(input(" Simulação máxima (anos) [30]: ") or 30)

return sat, F107, n_mc, anos

```

```

def parse_args():
    p = argparse.ArgumentParser(description="Simulador de Decaimento Orbital")
    p.add_argument("--preset", choices=list(PRESETS), help="Preset de satélite")
    p.add_argument("--massa", type=float, default=800)
    p.add_argument("--area", type=float, default=2.0)
    p.add_argument("--cd", type=float, default=2.2)
    p.add_argument("--altitude", type=float, default=500, help="km")
    p.add_argument("--F107", type=float, default=150.0)
    p.add_argument("--mc", type=int, default=200, help="Amostras MC")
    p.add_argument("--anos", type=float, default=30.0)
    p.add_argument("--sem-graficos", action="store_true")
    p.add_argument("--interativo", action="store_true")
    return p.parse_args()

```

```
#
```

```
# MAIN
```

```
#
```

```

def main():
    args = parse_args()

```

```

# Configuração
if args.interativo or len(sys.argv) == 1:
    sat, F107, n_mc, anos = entrada_interativa()
else:
    if args.preset:
        sat = deepcopy(PRESETS[args.preset])
    else:
        sat = Satellite(
            massa = args.massa,
            area = args.area,
            cd = args.cd,
            altitude = args.altitude * 1000,
        )
    F107 = args.F107
    n_mc = args.mc
    anos = args.anos

mostrar = not getattr(args, "sem_graficos", False)

print(f"\n Coeficiente balístico: {sat.bc:.2f} kg m-2")

# — Simulação determinística —————
print("\n ► Simulando decaimento orbital...")
sim = Simulador(sat, F107)
df_sim = sim.simular(anos_max=anos)
print(f" {len(df_sim)} pontos | "
      f"reentrada em {df_sim['tempo_anos'].iloc[-1]:.3f} anos")

if mostrar:
    Visualizador.decaimento(df_sim, f"Decaimento Orbital — {sat.altitude/1000:.0f} km")
    Visualizador.densidade_atmosferica()

# — Monte Carlo —————
print(f"\n ► Monte Carlo ({n_mc} amostras)...")
cfg_mc = ConfigMC(n_amostras=n_mc, anos_max=anos)
df_mc = monte_carlo(sat, F107, cfg_mc)
print(f" Concluído. Amostras válidas: {len(df_mc)}")

if mostrar:
    Visualizador.monte_carlo(df_mc)

# — Predição —————
print("\n ► Treinando preditor polinomial...")
preditor = PreditorDecaimento(grau=4)
r2 = preditor.treinar(df_sim)
print(f" R² = {r2:.6f}")

alt_query = sat.altitude / 1000.0

```

```
pred = preditor.prever(alt_query)
print(f"    Predição para {alt_query:.0f} km: {pred:.3f} anos")
```

```
# — Relatório —————
gerar_relatorio(df_sim, df_mc, sat, F107)
```

```
if __name__ == "__main__":
    main()
```